

SEÇÃO 1 (50 min) — "Como um modelo aprende?"

Slide 1 — Capa (1 min)

- Título: Aula 3 — Modelagem: ML, Deep Learning e LLMs
- Subtítulo: "Modelos não são mágica — são otimização."

Slide 2 — Mapa da aula (2 min)

- Diagrama simples em duas colunas:
 - **Seção 1:** Como modelos aprendem · Seleção · Treinamento · Overfitting
 - **Seção 2:** Transfer learning · Embeddings · LLMs · RAG · Prática
- Serve como orientação — o aluno sabe onde está a cada momento

Slide 3 — Seleção de algoritmos: o mapa mental (10 min)

- Uma única imagem de decisão com três perguntas:
 1. *Os dados são tabulares e rotulados?* → ML clássico
 2. *São imagens, áudio ou sequências longas?* → Deep Learning
 3. *É linguagem, código ou raciocínio?* → LLM
- Abaixo, uma tabela mínima (3 linhas × 4 colunas):

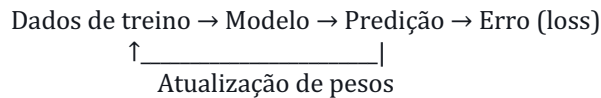
Tipo	Exemplos	Quando usar	Custo
ML clássico	XGBoost, Random Forest	Dados tabulares, interpretabilidade	Baixo
Deep Learning	CNN, Transformer	Volume alto, padrões complexos	Médio–alto
LLM	GPT, Claude, Gemini	Linguagem, raciocínio, geração	API ou alto

- **Mensagem principal:** "A escolha certa é a mais simples que resolve o problema."
- Tempo: professor explora cada linha com um exemplo real do contexto dos alunos

Slide 4 — Treinamento e validação: o ciclo (12 min)

- Diagrama de uma página com dois elementos principais:

Esquerda — O loop de treinamento:



Direita — A divisão dos dados:

- Treino / Validação / Teste — proporção e propósito de cada um
- Conceito de *data leakage* em uma linha: *"Se o modelo viu o dado de teste, a métrica mente."*
- **Demo ao vivo (sugerida):** abrir um notebook, mostrar curva de loss de treino e validação em tempo real com um dataset simples (sklearn ou Keras)
- Foco da discussão: *"Por que a loss de validação sobe mesmo com a de treino caindo?"* — leva naturalmente ao próximo slide

Slide 5 — Overfitting e Underfitting (10 min)

- Três curvas clássicas em uma figura (loss × épocas ou complexidade × erro):
 - **Underfitting:** erro alto em treino e validação
 - **Ideal:** erro baixo em ambos, curvas próximas
 - **Overfitting:** erro baixo em treino, alto em validação — curvas divergindo
- Causas e remédios em duas colunas simples:

Problema	Causa comum	Remédio
Underfitting	Modelo simples demais · pouco treino	Mais capacidade · mais épocas
Overfitting	Modelo complexo demais · poucos dados	Dropout · regularização · mais dados

- Pergunta para a turma: *"Qual é mais perigoso em produção? Por quê?"*
- Resposta esperada: overfitting — o modelo parece bom, mas falha no mundo real

Slide 6 — Pausa ativa (5 min)

- Uma pergunta única no slide, sem mais texto:

"Você tem um modelo com 98% de acurácia no treino e 71% no teste. O que você faz primeiro?"

- Discussão em duplas por 2 minutos → 3 minutos de plenária
 - Objetivo: fixar overfitting antes de avançar; revelar que a resposta começa nos dados, não no modelo
-

Slide 7 — Transição para Seção 2 (1 min)

- Frase única centralizada:

"Treinar do zero é caro. A maior parte dos modelos úteis não começa do zero."

SEÇÃO 2 (50 min) — "Modelos modernos e uso prático"

Slide 8 — Transfer Learning (8 min)

- Metáfora visual simples: duas colunas
 - **Sem transfer learning:** aprender a jogar xadrez sem saber as regras → do zero
 - **Com transfer learning:** saber xadrez e aprender damas → reaproveitamento
- Diagrama de três caixas verticais:

Modelo pré-treinado (ImageNet, Common Crawl, etc.)

↓ congela camadas anteriores

Fine-tuning nas camadas finais

↓

Modelo adaptado ao seu problema

- Casos concretos:
 - BERT fine-tuned para análise de sentimento
 - ResNet fine-tuned para detecção de defeitos em manufatura
 - **Mensagem:** *"Você herda bilhões de parâmetros já treinados — o custo é seu fine-tuning."*
-

Slide 9 — Fine-tuning vs. Prompting (8 min)

- Tabela de decisão direta:

	Fine-tuning	Prompting
Quando	Comportamento muito específico · dados proprietários	Maioria dos casos · prototipagem
Custo	Alto (GPU, dados rotulados)	Baixo (API)
Controle	Alto	Médio
Velocidade para produção	Semanas	Horas
Risco	Catastrophic forgetting	Prompt injection

- Regra prática: *"Comece sempre com prompting. Só faça fine-tuning quando o prompt não for suficiente."*
 - Exemplo ao vivo: mesmo problema resolvido com prompt bem escrito vs. ideia de fine-tuning — o professor mostra qual entrega mais valor mais rápido
-

Slide 10 — Embeddings: o que são e por que importam (8 min)

- Definição visual: texto/imagem/áudio → vetor numérico em espaço de alta dimensão
 - Figura mínima: três palavras ("rei", "rainha", "homem") como pontos em espaço 2D — distância semântica visível
 - Aplicações práticas:
 - Busca semântica (encontrar documentos similares)
 - Recomendação (usuários próximos no espaço = gostos parecidos)
 - RAG (recuperar chunks relevantes por similaridade)
 - **Por que engenheiros precisam saber:** embeddings são a interface entre dados não estruturados e sistemas de IA — aparecem em toda pipeline moderna
 - Ferramenta: `text-embedding-3-small` (OpenAI), `embed-v3` (Cohere), modelos locais via `sentence-transformers`
-

Slide 11 — LLMs: uso via API e RAG (10 min)

- Slide em duas metades:

Metade superior — Uso de APIs de LLM:

- Diagrama de três componentes: Sistema de prompt → Modelo (API) → Resposta
- Conceitos que o engenheiro precisa dominar:
 - Temperature e top-p (controle de aleatoriedade)
 - Tokens e janela de contexto (custo e limite)
 - System prompt vs. user prompt

Metade inferior — RAG:

- Diagrama do fluxo RAG em quatro passos:

Pergunta do usuário
↓
Embedding da pergunta → Busca vetorial no índice
↓
Chunks relevantes recuperados
↓
LLM gera resposta com contexto injetado

- Quando usar RAG: *"O modelo não sabe sobre seus dados — RAG é a ponte."*
- Ferramentas: LangChain, LlamaIndex, pgvector, Pinecone

Slide 12 — Atividade prática (12 min)

- Enunciado direto:

Em duplas, usando um notebook preparado ou o playground de uma API de LLM:

Parte A (5 min): Faça duas chamadas à API — uma com temperature 0, outra com temperature 1. Compare as respostas para a mesma pergunta. O que muda?

Parte B (5 min): Implemente um RAG mínimo: dado um parágrafo de contexto, faça o modelo responder uma pergunta usando apenas aquele contexto. Observe o que acontece se o contexto não contém a resposta.

Discussão (2 min): Que problemas de engenharia você já consegue enxergar nesse fluxo?

- Objetivo: tornar concretos os conceitos de temperatura, contexto e RAG antes da aula acabar
-

Slide 13 — Síntese e conexão com o curso (3 min)

- Retoma o mapa da Aula 1 (ciclo de vida): destaca onde a Aula 3 se encaixa — entre dados preparados e deploy
- Cinco ideias para levar:
 1. Escolha o modelo mais simples que resolve o problema
 2. Overfitting em produção é silencioso — sempre monitore a validação
 3. Transfer learning e prompting antes de treinar do zero
 4. Embeddings são a fundação de buscas e RAG modernos
 5. RAG resolve o problema do conhecimento privado sem fine-tuning